SOLID Principles – Introduction

In Java, **SOLID principles** are a set of **object-oriented design guidelines** used to design **robust, maintainable, and scalable software systems**. These principles focus on how classes and objects should be structured to reduce tight coupling and improve code quality.

The SOLID principles were **popularized by Robert C. Martin**, also known as **Uncle Bob**. Together, these five principles have significantly influenced modern object-oriented programming and have transformed the way software is designed and written.

By following SOLID principles, developers can build software that is:

- **Modular**
- **Easy to understand**
- **Easy to debug**
- **Easy to extend**
- **Easy to refactor**
- **Less error-prone during changes**

Overall, SOLID principles help create systems that can **adapt to changing requirements** while remaining **stable and reliable in production environments**.

What does SOLID stand for?

Letter	Principle
S	Single Responsibility Principle
O	Open / Closed Principle
L	Liskov Substitution Principle
I	Interface Segregation Principle
D	Dependency Inversion Principle

Open Close Principle (OCP)

Definition:
Software entities (classes, modules, functions) should be OPEN for extension but CLOSED for modification.

What does this mean?

- ☑ You should be able to **add new behavior**
- ✗ Without **changing existing, tested code**

Why OCP is Important?

- **Prevents breaking already working code**
- **Reduces regression bugs**

- Improves scalability and maintainability
- Encourages clean architecture
- Improve Extensibility

❌ Wrong Design (Violates OCP)

```
class DiscountService {
double getDiscount(String type) {
if (type.equals("STUDENT")) {
                return 10;
            }
           else if (type.equals("EMPLOYEE")) {
                return 20;
            }
return 0;
    }
}
```

🚨 Problems with this approach

- Every **new discount type** requires:
  - Modifying the existing class
  - Adding more if-else or switch statements

📌 **Example Problem**

If tomorrow we add SENIOR_CITIZEN, we must **edit this class again** ❌

☑ Correct Design (Follows OCP using Polymorphism)

Step 1: Create an abstraction

```
interface Discount {
   double applyDiscount();
}
```

Step 2: Create concrete implementations

```
class StudentDiscount implements Discount {
   public double applyDiscount() {
      return 10;
   }
}

class EmployeeDiscount implements Discount {
   public double applyDiscount() {
      return 20;
   }
}
```

🔁 **Adding New Discount (No Modification Required)**

```
class SeniorCitizenDiscount implements Discount {
   public double applyDiscount() {
      return 30;
   }
}
```

👉 **No changes** to DiscountService

👉 **No risk** to existing functionality

Step 3: Use abstraction in service class

```
class DiscountService {
double calculateDiscount(Discount discount) {
      return discount.applyDiscount();
   }
}
```